ALGORITHM

Smart Traffic Signal Scheduling System Using Circular Linked List

Algorithm: Circular Traffic Signal Scheduling

1. Start.
2. Define a node containing the signal name, green-light duration, and a pointer named **next**.
3. Initialize the circular linked list as empty.
4. Read the number of signals and the name and duration for each signal.
5. Create a node for each signal and link the nodes in the required order.
6. Connect the **next** pointer of the last node to the first node to make the list circular.
7. Set the current pointer to the first node.
8. Display the current signal as GREEN and display its configured duration.
9. Simulate the green-light interval using a timer or delay.
10. Display the current signal as RED (or inactive) and advance the current pointer to **current->next**.
11. If the user has not requested stop, repeat Steps 8–10. Otherwise, stop the schedule.
12. End.

Important condition

If the list contains at least one node, the final node must point to the head node. During scheduling, traversal uses current = current->next; it should not stop merely because the current pointer returns to the head, since that return is the intended circular behavior.

Example sequence

North Road (30 seconds) → East Road (25 seconds) → South Road (30 seconds) → West Road (20 seconds) → back to North Road.

Note: The durations are sample values and can be changed according to project requirements.